
KTU
NOTES

EET 303 MICROPROCESSORS AND EMBEDDED SYSTEMS

MODULE-1

Syllabus

Internal architecture of 8085 microprocessor–Functional block diagram

Instruction set-Addressing modes - Classification of instructions - Status flags.

Machine cycles and T states – Fetch and execute cycles- Timing diagram for instruction and data flow.

1.1 8085 Microprocessor

Features of 8085:

- It is an 8 bit microprocessor.
- It is manufactured with N-MOS technology.
- It has 16-bit address bus and hence can address up to 2^{16} = 65536 bytes (64KB) memory locations through A_0-A_{15}.
- It has 8-bit data bus.
- It supports external interrupt request.
- It requires a signal +5V power supply and operates at 3.2 MHZ single phase clock.
- It is enclosed with 40 pins, DIP (Dual in line package).

Architecture of 8085

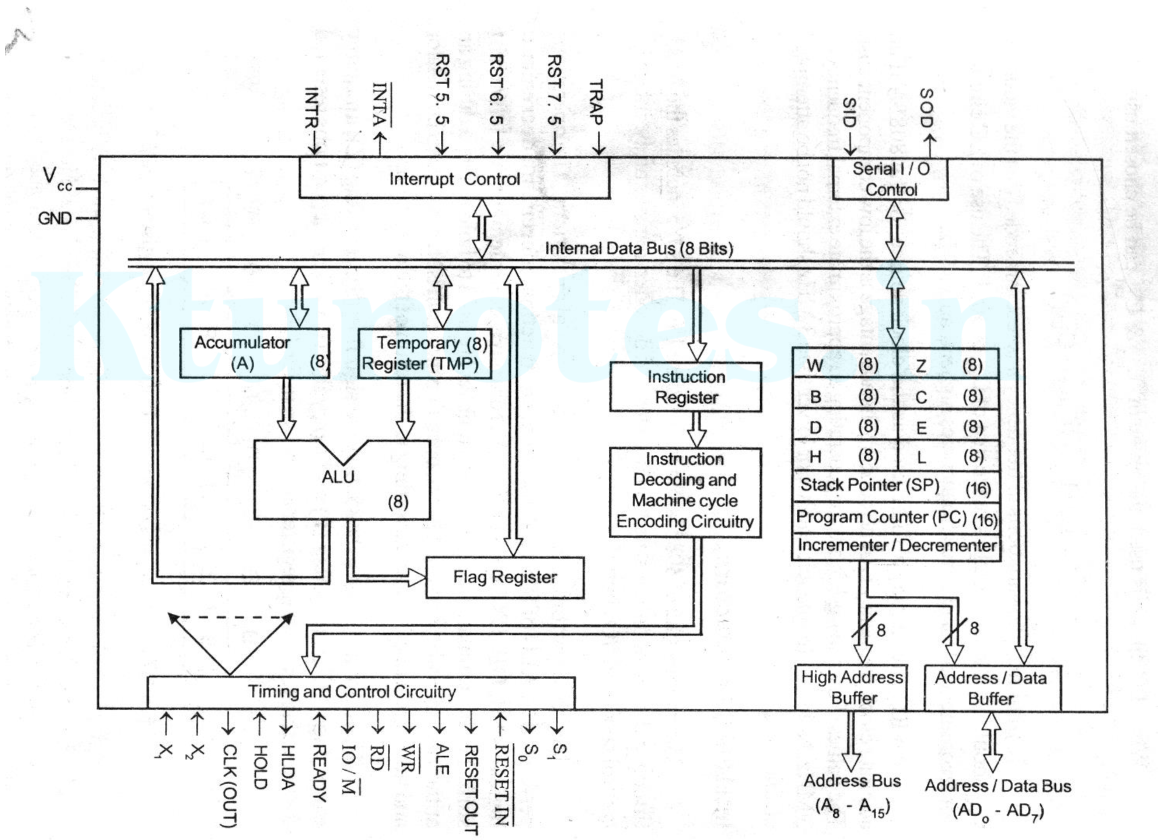


Figure 1.2 Internal architecture of 8085

The architecture of 8085 is shown in figure 1.2. The internal architecture of 8085 includes the ALU, register array, timing and control unit, instruction register and decoder, interrupt control and serial I/O control.

ALU:

Functions of ALU:

- It performs arithmetic operations like ; addition, subtraction, increment, etc.
- It performs logical operations like ; AND ing, OR ing, X-OR ing, NOT etc.
- It accepts operands from accumulator and temporary register.

- It stores the result in accumulator.
- It provides states of result to the flag register.

Register Array:

1. **General Purpose Registers:**

- The 8085/8080A has six general-purpose registers to store 8-bit data; these are identified as B,C,D,E,H, and L.
- They can be combined as register pairs - BC, DE, and HL - to perform some 16-bit operations.
- The temporary registers W and Z are intended for internal use of the processor and it cannot be used by the programmer.

2. **Accumulator:**

- The accumulator is an 8-bit register that is a part of arithmetic/logic unit (ALU). This register is used to store 8-bit data and to perform arithmetic and logical operations.
- The result of an operation is stored in the accumulator.
- The accumulator is also identified as register A.

3. **Flag Register/ Status Flags:**

   - Flag is a flip-flop which changes its status according to the result stored in the accumulator.
   - Flag register is also known as status register.
   - 8085 has an 8-bit Flag register with 5 active flags. They are called Zero (Z), Carry (CY), Sign (S), Parity (P), and Auxiliary Carry (AC) flags.
   - The bit position of the flip flop in flag register is:

D_7	D_6	D_5	D_4	D_3	D_2	D_1	D_0
S	Z		AC		P		CY

Sign Flag- If D7 of the result is 1 then sign flag is set otherwise reset. As we know that a number on the D7 always decides the sign of the number.

Zero Flag (Z)-If the result stored in an accumulator is zero then this flag is set otherwise it is reset.

Auxiliary carry Flag (AC)-If any carry goes from D3 to D4 in the result, then it is set otherwise it is reset.

Parity Flag (P)-If the no of 1's in the result stored in the accumulator is even, then it is set otherwise it is reset for **the** odd.

Carry Flag (C)-If the result stored in an accumulator generates a carry in its final output then it is set otherwise it is reset.

4. **Program Counter (PC):**

   - This is a 16-bit register used to hold the address of the next instruction to be executed.
   - When a byte (machine code) is being fetched, the program counter is incremented by one to point to the next memory location.

5. **Stack Pointer (SP):**

- The stack pointer is also a 16-bit register used as a memory pointer.
- It points to a memory location in R/W memory, called the stack.
- The beginning of the stack is defined by loading 16-bit address in the stack pointer.

6. Instruction Register (IR):

- It is a 8-bit register. An instruction fetched from memory is temporarily stored in Instruction Register before decoding.
- The register is not accessible to user.
- Instruction register holds the opcode of instruction that is to be decoded and executed.

Instruction Decoder:

- Instruction decoder takes bits stored in the instruction register and decodes it and tells to CPU what it need to do for it and enable the components for the operation.
- Simply, instruction decoder is like a dictionary. It tells the meaning of the instruction.

Timing and Control Unit:

- It provides timing and control signal to the microprocessor to perform the various operations.
- It has three control signals. It controls all external and internal circuits.
- It operates with reference to clock signal.
- It synchronizes all the data transfers.

Serial Input Output Control:

- There are two pins in this unit. This unit is used for serial data communication.

Interrupt Unit:

- Interrupt is a mechanism by which an I/O or an instruction can suspend the normal execution of processor and get itself serviced.
- There are 6 interrupt pins in this unit. Generally an external hardware is connected to these pins.
- These pins provide interrupt signal sent by external hardware to microprocessor and microprocessor sends acknowledgement for receiving the interrupt signal.

1.2 Instructions

- An instruction is a command to the microprocessor to perform a given task on a specified data. Each instruction has two parts: one is task to be performed, called the **operation code (op-code**), and the second is the data to be operated on, called the **operand**.

Instruction Set Classification:

- The entire group of instructions is called the **instruction set**.
- These instructions can be classified into the following five functional categories: data transfer (copy) operations, arithmetic operations, logical operations, branching operations, and machine-control operations.

1. Data Transfer (Copy) Instructions:

- This group of instructions copy data from a location called a source to another location called a destination, without modifying the contents of the source.
- The various types of data transfer (copy) are listed below together with examples of each type:

Types	Examples
1. Between Registers.	1. Copy the contents of the register B into register D. **MOV D,B**
2. Specific data byte to a register or a memory location.	2. Load register B with the data byte 32H. **MVI B, 32H**
3. Between a memory location and a register.	3. From a memory location 2000H to register B. **MOV B,M**
4. Between an I/O device and the accumulator.	4.From an input keyboard to the accumulator. **IN PORT ADDRESS**

2. Arithmetic Instructions

These instructions perform arithmetic operations such as addition, subtraction, increment, and decrement.

***Addition*:-**

- Any 8-bit number, or the contents of a register or the contents of a memory location can be added to the contents of the accumulator and the sum is stored in the accumulator.
- No two other 8-bit registers can be added directly (e.g., the contents of register B cannot be added directly to the contents of the register C).
- Eg:- ADD B,ADI 09

Subtraction:-

- Any 8-bit number, or the contents of a register, or the contents of a memory location can be subtracted **from the contents of the accumulator** and the **results stored in the accumulator**.
- The subtraction is performed in 2's compliment, and the results if negative, are expressed in 2's complement. **No two other registers** can be subtracted directly.
- Eg:- SUB C, SBI 08

***Increment/Decrement*:-**

- The 8-bit contents of a register or a memory location can be incremented or decrement by 1. Similarly, the 16-bit contents of a register pair (such as BC) can be incremented or decrement by 1.
- Eg:- INR D, DCR D

3. **Logical Instructions**

These instructions perform various logical operations with the contents of the accumulator.

AND, OR Exclusive-OR: -

- Any 8-bit number, or the contents of a register, or of a memory location can be logically ANDed, ORed, or Exclusive-ORed with the contents of the accumulator.
- The results are stored in the accumulator.
- Eg:- ANA D, ORA C

Rotate:-

- Each bit in the accumulator can be shifted either left or right to the next position.
- Eg:- RLC,RAL,RRC,RAR

Compare:-

- Any 8-bit number or the contents of a register, or a memory location can be compared for equality, greater than, or less than, with the contents of the accumulator.
  Eg:- CMP B, CPI 08

Complement:-

- The contents of the accumulator can be complemented. All 0s are replaced by 1s and all 1s are replaced by 0s.
  Eg:- CMA

4. Branching Instructions

This group of instructions alters the sequence of program execution either conditionally or unconditionally.

Jump:-

- Conditional jumps are an important aspect of the decision-making process in the programming.
- These instructions test for a certain conditions (e.g., Zero or Carry flag) and alter the program sequence when the condition is met.
  Eg:- JC address ,JNC address, JZ address
- In addition, the instruction set includes an instruction called *unconditional jump.*
  Eg:- JMP address

Call, Return, and Restart:-

- These instructions change the sequence of a program either by calling a subroutine or returning from a subroutine.
- The conditional Call and Return instructions also can test condition flags.
  Eg:- CALL address, RET

5. Machine Control Instructions

These instructions control machine functions such as Halt, Interrupt, or do nothing.

Eg:- i) SIM ii) RIM iii) HLT

1.3 Addressing Modes

- The method by which the address of source of data or the address of destination of result is given in the instruction is called Addressing Modes.
- The term addressing mode refers to the way in which the operand of the instruction is specified.
- Intel 8085 uses the following addressing modes:
  1. Direct Addressing Mode
  2. Register Addressing Mode
  3. Register Indirect Addressing Mode
  4. Immediate Addressing Mode
  5. Implicit Addressing Mode

Immediate Addressing

- In immediate addressing mode, the data is specified in the instruction itself.
- The data will be a part of the program instruction.
- All instructions that have 'I' in their mnemonics are of Immediate addressing type.

Eg. MVI B, $3E_H$ - Move the data $3E_H$ given in the instruction to B register.

Direct Addressing

- In direct addressing mode, the address of the data is specified in the instruction. The data will be in memory.
- This type of addressing can be identified by 16-bit address present in the instruction.

Eg. LDA 1050_H - Load the data available in memory location 1050_H in accumulator.

Register Addressing

- In register addressing mode, the instruction specifies the name of the register in which the data is available.
- This type of addressing can be identified by register names (such as 'A', 'B', …) in the instruction.

Eg. MOV A, B -Move the content of B register to A register.

Register Indirect Addressing

- In register indirect addressing mode, the instruction specifies the name of the register in which the address of the data is available.
- Here the data will be in memory and the address will be in the register pair.
- This type of addressing can be identified by letter 'M' present in the instruction.

Eg. MOV A, M - The memory data addressed by HL pair is moved to A register.

Implied Addressing

- In implied addressing mode, the instruction itself specifies the type of operation and location of data to be operated.
- This type of instruction does not have any address, register name, immediate data specified along with it.

Eg. CMA - Complement the content of accumulator.

1.4 The 8085 Machine Cycles and Timings

Timing diagram:

- It is the graphical representation of process in steps with respect to time.
- The timing diagram represents the clock cycle and duration, delay, content of address bus and data bus, type of operation ie. Read/write/status signals.

T-state:

- T-state is the time corresponding to one clock period. It is a basic unit used to calculate the time taken for execution of instructions and programs in a processor.

Machine Cycle:

- A machine cycle is the time required to complete one operation of accessing the memory, I/O or acknowledge an external signal or request.
- Usually machine cycle consists of 3 to 6 T-states.
- The different types of machine cycle available in 8085 microprocessor are:
  - Opcode Fetch
  - Memory Read
  - Memory write

- I/O Read
- I/O Write
- INTR Acknowledge
- Bus Idle

Instruction Cycle:

- An instruction is a command given to the microprocessor to perform a specific operation on the given data.
- Sequence of instructions written for a processor to perform a particular task is called a program.
- Program & data are stored in the memory.
- The microprocessor fetches one instruction from the memory at a time & executes it. It executes all the instructions of the program one by one to produce the final result.
- The necessary steps that a microprocessor carries out to fetch an instruction & necessary data from the memory & to execute it constitute an instruction cycle.
- In other words, an instruction cycle is defined as the time required completing the execution of an instruction.
- An instruction cycle consists of a fetch cycle and an execute cycle. The time required to fetch an opcode (fetch cycle) is a fixed slot of time while the time required to execute an instruction (execute cycle) is variable which depends on the type of instruction to be executed.

  Instruction cycle(IC) = Fetch cycle(FC) + Execute cycle(EC)

1. Timing diagram of Memory Read Machine Cycle

The memory read machine cycle is executed by the processor to read a data byte from memory. The processor takes 3 T-states to execute this cycle. The timings of various signals during memory read cycle are shown below.

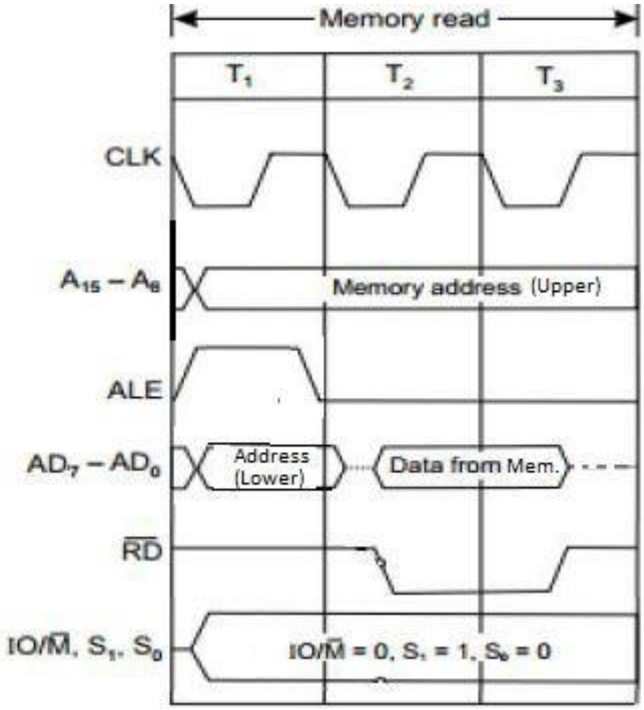


Fig 2.1: Timing diagram of memory read machine cycle

After the falling edge of T_1,

- The microprocessor outputs the low byte address on AD_0 – AD_7 lines and high byte address on A_8 to A_{15} lines.
- ALE is asserted high to enable the address latch.
- The other control signals are as follows
  $IO/\overline{M}$ =0, S_0=0, S_1=1.

In the second T-state (T_2),

- The memory is requested for read by asserting read line $\overline{RD}$ low.
- When read is asserted low, the memory is enabled for placing the data on the data bus.

In the third T-state T_3,

- Data from data bus are placed into the specified register (A,B, C, etc.) and raises $\overline{RD}$ so that memory is disabled.

2. Memory Write Machine cycle

This cycle is used for sending data from the registers of the microprocessor to the memory. The processor takes 3 T-states to execute this cycle. The timings of various signals during memory write cycle are shown below.

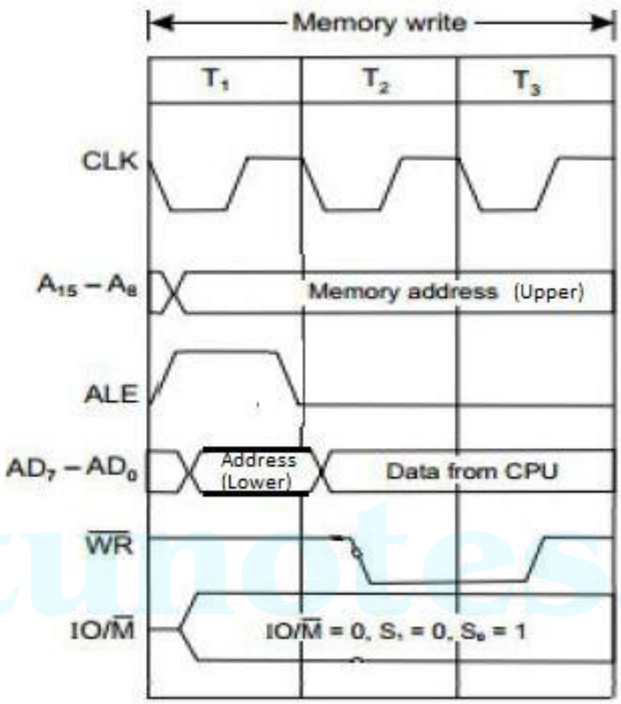


Fig 2.2: Timing diagram of memory write machine cycle

After the falling edge of T_1,

- The microprocessor outputs the low byte address on AD_0 – AD_7 lines and high byte address on A_8 to A_{15} lines.
- ALE is asserted high to enable the address latch.
- The other control signals are as follows
  $IO/\overline{M}$ =0, S_0=1, S_1=0.

In the second T-state (T_2),

- During this state the data to be written is placed on the Data bus.
- The write control signal $\overline{WR}$ goes low.

In the third T-state T_3,

- The data which was placed on the data bus is now transferred to the specific memory location.
- In the middle of this state the $\overline{WR}$ goes high and disables the memory.

3. I/O Read Machine Cycle

In the I/O operations, since the address of I/O ports is 8-bits, external latching using ALE is not necessary. Hence in I/O operations, the address is duplicated and is available on the address bus till the end of the machine cycle.

The I/O Read cycle is executed by the processor to read a data byte from I/O port. The processor takes 3 T-states to execute this machine cycle. The timings of various signals during this machine cycle are shown in figure below.

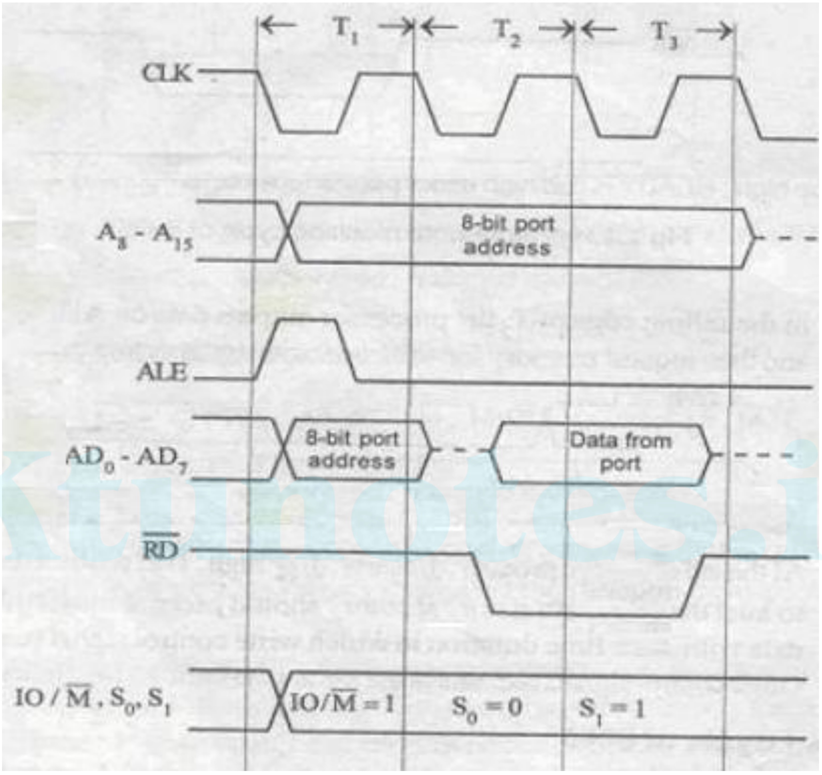


Fig 2.3: Timing diagram of I/O read machine cycle

At the falling edge of T_1,

- The microprocessor outputs the 8 bit port address on both the low order address lines (AD_0-AD_7) and high order address lines (A_8 to A_{15}).
- ALE is asserted high to enable the address latch. The other control signals are asserted as follows.
  $IO/\overline{M}$ =1, S_0=0 and S_1=1. ($IO/\overline{M}$ is asserted high to indicate I/O read operation).

In the second T-state (T_2)

- The I/O device is requested for read by asserting read line $\overline{RD}$ low.
- When RD is asserted low, the I/O port is enabled for placing the data on the data bus.

At the end of T_3,

- The data is transferred into microprocessor. The read signal is asserted high. Other control signals remains in the same state until the next machine cycle.

4. I/O Write Machine Cycle:

The I/O write cycle is executed by the processor to send a data byte from processor to I/O port. The processor takes 3 T-states to execute this machine cycle. The timings of various signals during this machine cycle are shown in figure below.

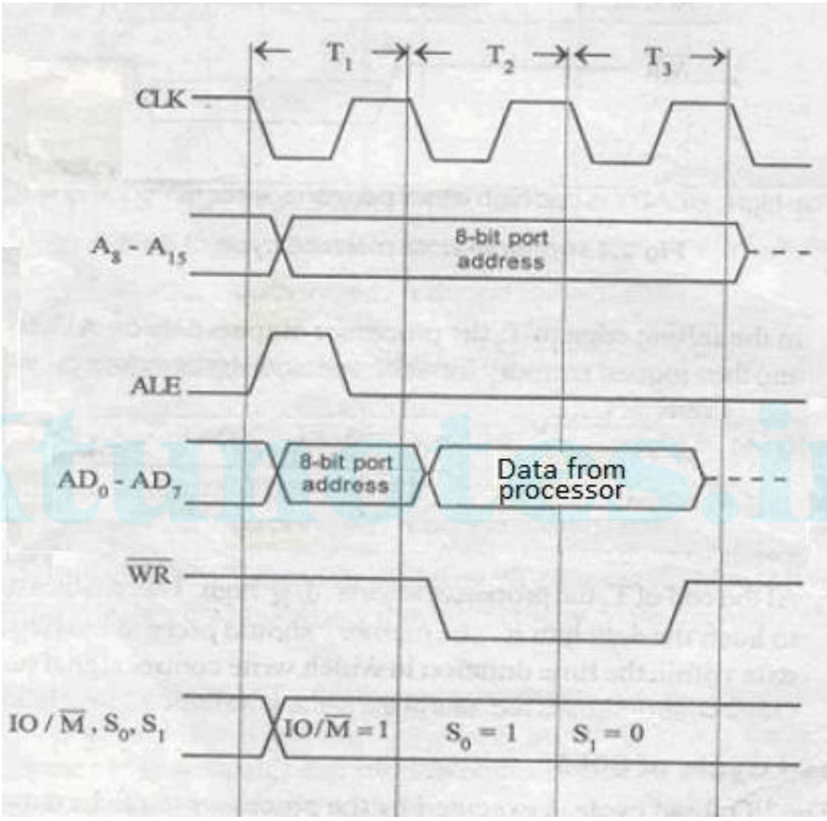


Figure 2.4 : Timing diagram of I/O write machine cycle

At the falling edge of T_1,

- The microprocessor outputs the 8 bit port address on both the low order address lines (AD_0-AD_7) and the high order address lines.
- ALE is asserted high to enable the address latch. The other control signals are asserted as follows.
  $IO/\overline{M}$ =1, S_0=1 and S_1=0. ($IO/\overline{M}$ is asserted high to indicate I/O read operation).

In the second T-state (T_2)

- In the falling edge of T_2 the processor outputs data on AD_0-$\overline{AD_7}$ lines and then request I/O port for write operation by asserting the write control signal $\overline{WR}$ to low.

At the end of T_3,

- The data which was placed on the data bus in the previous state is now transferred to the I/O device.
- In the middle of this state the $\overline{WR}$ goes high and disables the I/O device.

5. Opcode Fetch Machine Cycle of 8085:

Each instruction of the processor has one byte opcode. The opcodes are stored in memory. The opcode fetch machine cycle is executed by the processor to fetch the opcode from memory. Hence, every instruction starts with opcode fetch machine cycle.

The time taken by the processor to execute the opcode fetch cycle is either 4T or 6T. In this time, the first 3T-states are used for fetching the opcode from memory and the remaining T-states are used for internal operations by the processor. The timings of various signals during opcode fetch cycle is shown as:

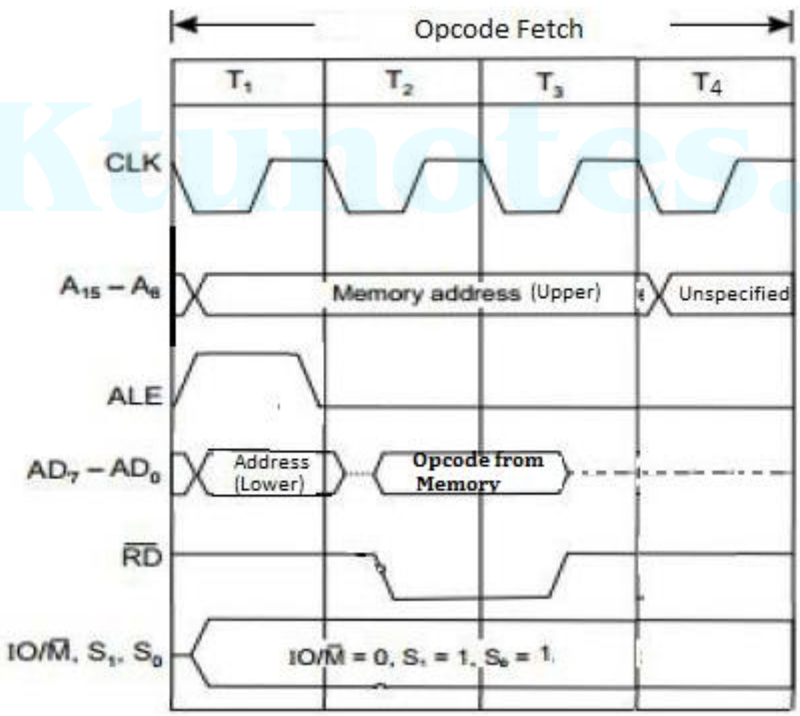


Figure 2.5: Timing diagram of opcode fetch machine cycle

After the falling edge of T_1,

- The microprocessor outputs the low byte address on AD_0 – AD_7 lines and high byte address on A_8 to A_{15} lines.
- ALE is asserted high to enable the address latch.
- The other control signals are as follows
  $IO/\overline{M}$ =0, S_0=1, S_1=1.

In the second T-state (T_2),

- The memory is requested for read by asserting read line $\overline{RD}$ low.
- When read is asserted low, the memory is enabled for placing the data on the

data bus. In the third T-state T_3,

- Data from data bus are placed into the Instruction Register and raises $\overline{RD}$ so that memory is disabled.

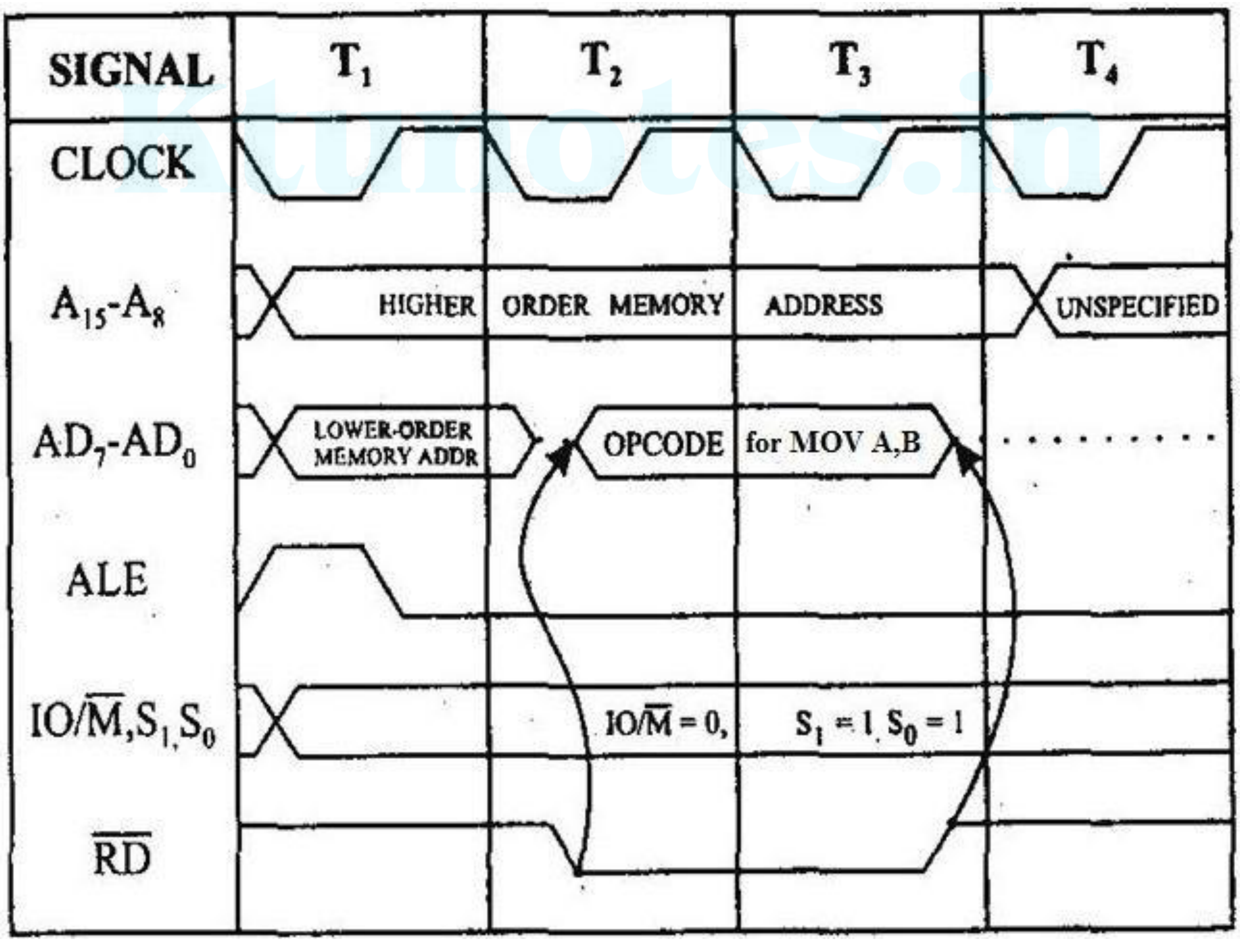
In the fourth T-state T_4,

- This t-state is used for internal operations by the processor like decoding.

---Exercise---

1. Timing Diagram for MOV A,B

The instruction MOV A,B is a 1-byte instruction. Microprocessor takes only one machine cycle (op-code fetch) to complete instruction. Hence, hex code for MOV A,B is passed to the microprocessor.



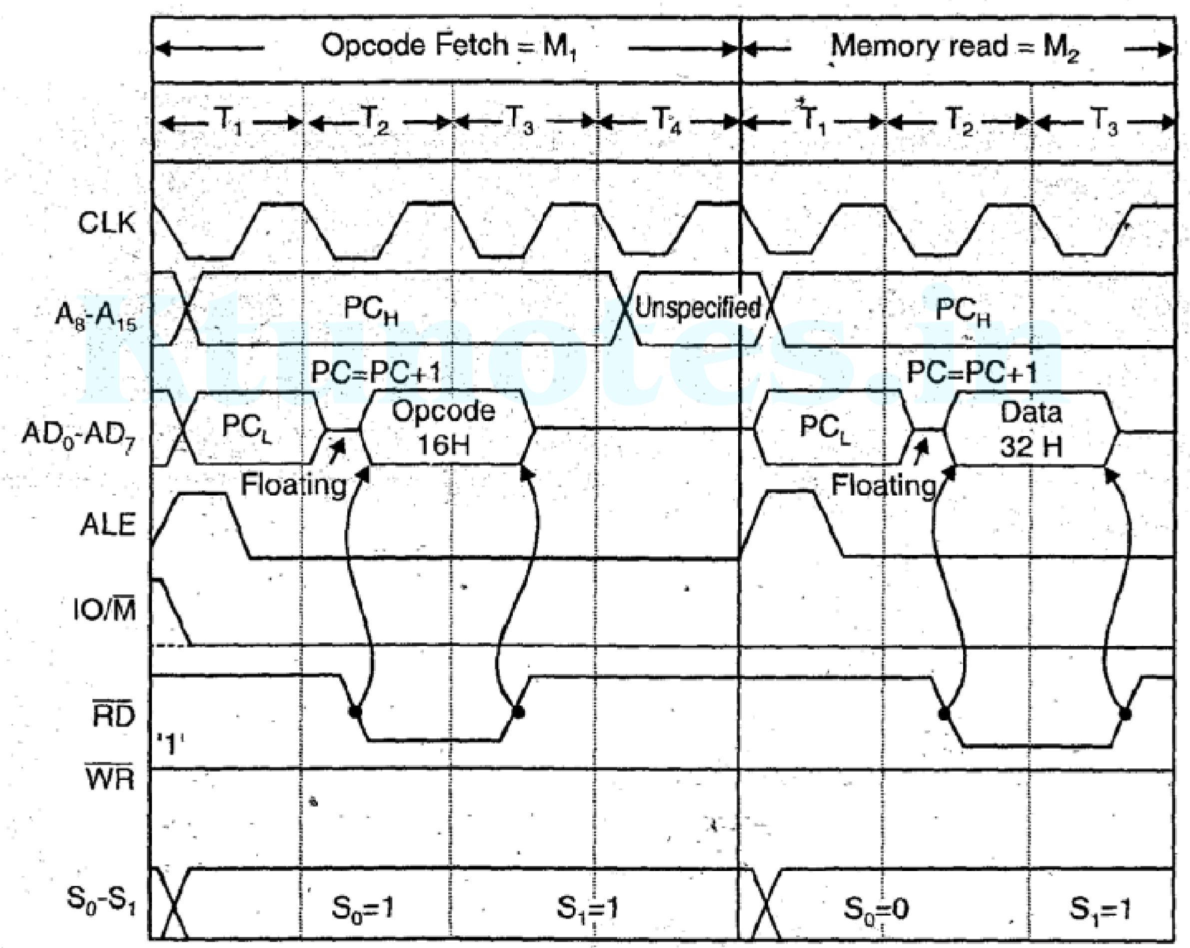
2. Draw the 8085 timing of execution of the 2 byte instruction MVI A, 32H (load the accumulator with the data 32 H) store in location as follows

Memory location	Machine Code	Mnemonics
2000	3E	MVIA,32H
2000	32	

Ans.

This is a 2 byte instruction so it requires 2 machine cycles to fetch the instruction

1. Op code fetch and
2. Memory read



Timing diagram of MVI A, 32H

8085 µP Instructions

- An **Instruction** is a binary pattern designed inside a µP to perform a specific function.

- The entire group of instructions, called the **instruction set** determines what functions the microprocessor can perform.

- The entire 8085 Instruction set can be classified into 5 functional categories.

Classification of 8085 Instructions

- Data Transfer Instructions
- Arithmetic Instructions
- Logic Instructions
- Branch Instructions
- Machine Control Instructions

Data Transfer Instructions

- This group of instructions copies data from a location called source to another location called destination without modifying the contents of the source.

- While transferring the data, the source contents are retained without any changes.

- Various types of data transfer operations are possible in 8085 µP

Types of Data Transfer Operations

Sl No	Types	Examples
1	Between Registers	Copy content of register B into register D
2	Specific data byte to a register	Load register B with data byte 33_H
3	Between memory location and a register	Copy from location 2000_H to register B
4	Between an I/O device and Accumulator	From an Input Keyboard to the Accumulator

Arithmetic Instructions

- These instructions perform arithmetic operations

1. **Addition** – Can perform 8-bit and 16-bit addition. 8-bit addition can be performed only by keeping one number in the Accumulator. The other number can be can be stored in any register or memory location. The sum is stored in Accumulator. (special case – DAD instruction)

2. **Subtraction** – Can perform 8-bit and 16-bit subtraction. Any 8-bit number or contents of a register or the contents of a memory location can be subtracted from the contents of accumulator and result is stored in Accumulator.

Arithmetic Instructions

- Subtraction in 8085 µP is performed in 2’s complement and result, if negative, is expressed in 2’s complement.
- 2’s complement method helps to reduce hardware requirement. It has unique representation of zero. For an n-bit word range of numbers in this method is $-(2^{n-1})$ to $+(2^{n-1}-1)$.

- **Increment/Decrement** – The 8-bit contents of a register or a memory location and a 16-bit content of a register pair can be incremented or decremented by 1.
- Increment/Decrement instructions does not require use of Accumulator register and can be performed in any one register or memory location.

Logical Instructions

- Performs logical operations with the Accumulator control

1. **AND, OR, X-OR** – Any 8-bit number (or contents of Reg. / Memory) can be logically ANDed, Ored or X-Ored with contents of Accumulator. Result stored in Accumulator.
2. **Rotate** – Each bit in Accumulator can be shifted either left or right to next position.
3. **Compare** - Any 8-bit number (or contents of Reg. / Memory) can be compared with contents of Accumulator (> = <)
4. **Complement** – The contents of Accumulator can be complemented (1→ 0 & 0 → 1)

Branching and Control Instructions

- These instructions alter the sequence of program execution either conditionally or unconditionally

1. **Jump** – Conditional Jumps are decision making instructions which checks for a certain condition (flags) and alter the program sequence if the condition is met. It also includes an unconditional jump instruction
2. **Call, Return and Restart** – These are used to change the sequence of program by calling a subroutine or returning from a subroutine. Generally used in unconditional manner but can also be used conditionally.

- Control instructions are used to control machine functions such as Halt, Interrupt or do-nothing(NOP).

More about Instructions

- An instruction is a command given to the µP to perform a given task on specified data.
- It has two parts
  - Operation Code (Opcode) : Task to be performed
  - Operand : Data to be operated on.

- The operand can be specified/given in various forms. The different ways of specifying the operand are called the addressing modes.
- Operands can be specified as 8-bit (or 16-bit) data, as internal register, a memory location, an address of location or even implicitly (hidden).

Instruction Word Size

- Depending on the word (or byte) size the 8085 instructions are classified into the following 3 groups.

1. 1-byte instructions
2. 2-byte instructions
3. 3-byte instructions

1-Byte Instructions

- It includes the opcode and operand in the same byte.
- Example

Task	Opcode	Operand	Binary Code	Hex Code
Copy contents of Accumulator to register C	MOV	C,A	0100 1111	$4F_H$
Add contents of register B to contents of accumulator	ADD	B	1000 0000	80_H
Complement each bit in accumulator	CMA		0010 1111	$2F_H$

- All these instructions are stored in 8-bit binary format and require only one memory location.
- The first operand is destination and the second is source.

2-Byte Instructions

- The first byte specifies the opcode and the second byte specifies the operand.

- Example

Task	Opcode	Operand	Binary Code	Hex Code
Load an 8-bit data (32_H) into Accumulator	MVI	A, 32_H	0011 1110 0011 0010	3E 32

- These instructions require two memory locations to store the binary codes.

3-Byte Instructions

- The first byte specifies the opcode and the following two bytes specifies the 16-bit memory address where the operand is stored. (In memory, first lower order address is stored)

- Example

Task	Opcode	Operand	Binary Code	Hex Code
Load contents of Memory location 2050H into accumulator	LDA	2050H	0011 1010 0101 0000 0010 0000	$3A_H$ 50_H 20_H

- These instructions require three memory locations to store binary codes.

Addressing Modes

- Various ways of specifying data/operand are called addressing modes.

- In any 8085 instruction, the programmer need to specify the following details,

  - Operation to be performed - Opcode
  - Source of Data
  - Destination of Result

  - Operand

- While giving instructions in 8085, the operand source and destination location need to be specified. Different ways in which the processor identifies the source and destination of data from an instruction are called addressing modes.

Addressing Modes

- The various addressing modes help us to understand how operands are identified in various instructions
- Generally, the source and destination of operands can be
  - Register
  - Memory Location
  - 8-bit number (through instruction or input device)

- There are five addressing modes available in 8085.

- Each addressing mode will have a specific way of providing/storing the data (operand).

Addressing Modes

- Immediate Addressing Mode
- Direct Addressing Mode
- Register Addressing Mode
- Register Indirect Addressing Mode
- Implicit Addressing Mode

Immediate Addressing Mode

- In this mode, data is specified with the instruction as operand.
- If the data is 8-bit, the instruction is of 2-byte and if the data is of 16-bit, the instruction is of 3-byte.
- Example,
  - MVI B, 45
  - LXI H, 3050
  - JMP 8004

Direct Addressing Mode

- In this mode, the data is available in a memory location and memory location is specified in the instruction.

- Here operand of the instruction consists of the memory address of data

- Example,
  – LDA 2050 – load contents of memory location 2050 to Accumulator
  – LHLD 9500 – load contents of 16-bit memory into HL pair

Register Addressing Mode

- In this mode, the data is available in a register and the register is specified in the instruction.

- Here operand of the instruction consists of the source or destination register.

- Example,
  - MOV A, B
  - ADD B
  - INR A

Register Indirect Addressing Mode

- In this mode, the data is available in a memory location and the address of the memory location is provided by the register pair

- The operand will be the register pair

- Example
  - MOV A,M – Move contents of the memory location pointed by HL pair registers
  - LDAX B – Move contents of B-C register pair to Accumulator

Implicit Addressing Mode

- In this mode, the operand is hidden and the data to be processed is available within the opcode

- Example,
  - CMA – Complement contents of Accumulator
  - RRC – Rotate Accumulator content right by 1bit
  - RLC – Rotate Accumulator content left by 1 bit

Data Format

- Data formats provide various binary representation of decimal numbers, letter and symbols that are required for programming.

1. ASCII Code – 7-bit binary code that represents numbers, letter and special characters.
2. BCD Code – Binary coded decimal system is used to represent decimal numbers in binary form.
3. Signed Integer – Used to represent positive and negative numbers. MSB is used to represent the sign of the number.
4. Unsigned Integer – Used to represent positive integers. All the bits are used to represent the number

8085 Instruction Set – Frequently Used

There are 74 different opcodes that results in 246 instructions. The most frequently used instructions are provided in the following slides.

The following notations are used in the description of instructions :

- R – 8085 8-bit Register (A, B, C, D, E, H & L)
- M – Memory location
- Rs – Register Source
- Rd – Register Destination
- Rp – Register Pair (BC, DE, HL, SP)

Complete instruction set is provided in the Appendix F

Data Transfer Instructions

Sl. No	Mnemonics	Example	Operation
1.1	MVI R, 8-bit	MVI B, 4F	Load 8- bit data in a register
1.2	MOV Rd, Rs	MOV B, C	Copy data from source register (Rs) to destination register (Rd)
1.3	LXI Rp, 16-bit	LXI B, 2050	Load 16-bit number to a register pair
1.4	OUT 8-bit	OUT 01	Send (write) data byte from A register to an output device
1.5	IN 8-bit	IN 07	Accept (read) data byte into A register from an input device
1.6	LDA 16-bit	LDA 2050	Copy data into A register from the memory location specified by 16-bit address

Data Transfer Instructions

l. No	Mnemonics	Example	Operation
1.7	STA 16-bit	STA 2070	Copy data from A register into the memory location specified by the address
1.8	LDAX Rp	LDAX B	Copy the data into A register from the memory specified by the address in the register pair
1.9	STAX Rp	STAX D	Copy the data from A register into the memory specified by the address in the register pair
.10	MOV R, M	MOV B, M	Copy the data byte into the specified register from the memory specified by the address in HL pair
.11	MOV M,R	MOV M, C	Copy the data byte from the specified register into the memory specified by the address in HL pair

Arithmetic Instructions

Sl. No	Mnemonics	Example	Operation
2.1	ADD R	ADD B	Add contents of the specified register to the contents of A register
2.2	ADD 8-bit	ADI 37	Add the data to the contents of A register
2.3	ADD M	ADD M	Add the contents of memory to A register. The memory address in HL register pair
2.4	SUB R	SUB C	Subtract contents of the specified register to the contents of A register
2.5	SUI 8-bit	SUI 7F	Subtract the data to the contents of A register
2.6	SUB M	SUB M	Subtract the contents of memory to A register. The memory address in HL register pair

Arithmetic Instructions

Sl. No	Mnemonics	Example	Operation
2.7	INR R	INR D	Increment the contents of the specified register
2.8	INR M	INR M	Increment the contents of the memory, the address of which is in HL pair
2.9	DCR R	DCR E	Decrement the contents of the specified register
2.10	DCR M	DCR M	Decrement the contents of the memory, the address of which is in HL pair
2.11	INX Rp	INX H	Increment the contents of a register pair
2.12	DCX Rp	DCX B	Decrement the contents of a register pair

Logic and Bit Manipulation Instructions

l. No	Mnemonics	Example	Operation
3.1	ANA R	ANA B	Logical AND the contents of register specified with contents of A register
3.2	ANI 8-Bit	ANI 2F	Logical AND 8-bit data with contents of A register
3.2	ANA M	ANA M	Logical AND the contents of memory with contents of A; The memory address in HL register pair
3.4	ORA R	ORA E	Logical OR the contents of register specified with contents of A register
3.5	ORI 8-bit	ORI 3F	Logical OR 8-bit data with contents of A register
3.6	ORA M	ORA M	Logical OR the contents of memory with contents of A; The memory address in HL register pair

Logic and Bit Manipulation Instructions

Sl. No	Mnemonics	Example	Operation
3.7	XRA R	XRA B	X-OR the contents of register specified with contents of A register
3.8	XRI 8-bit	XRI 6A	X-OR 8-bit data with contents of A register
3.9	XRA M	XRA M	X-OR the contents of memory with contents of A; The memory address in HL register pair
3.10	CMP R	CMP B	Compare contents of register specified with that of A register (>= <)
3.11	CPI 8-bit	CPI 4F	Compare data with contents of A register (> = <)

Branch Instructions

Sl. No	Mnemonics	Example	Operation
4.1	JMP 16-bit	JMP 2050	Change program sequence to specified address
4.2	JZ 16-bit	JZ 2080	Change program sequence to specified address if Zero flag is Set (1)
4.3	JNZ 16-bit	JNZ 2070	Change program sequence to specified address if Zero flag is reset (0)
4.4	JC 16-bit	JC 2055	Change program sequence to specified address if carry flag is Set (1)
4.5	JNC 16-bit	JNC 2030	Change program sequence to specified address if carry flag is reset (0)
4.6	CALL 16-bit	CALL 2075	Change program sequence to specified subroutine location
4.7	RET	RET	Return to Calling program after completing Subroutine

Machine Control Instructions

Sl. No	Mnemonics	Example	Operation
5.1	HLT	HLT	Stop processing and wait
5.2	NOP	NOP	Do not perform any operation

Instruction Cycle, Machine Cycle and T-State

- The general operation of 8085 µP consists of sequential execution of instructions stored in the R/W Memory.
- To differentiate **opcode** from **data**, 8085 µP assumes the first byte of hexadecimal numbers it fetch from memory as **opcode**.

- The execution of each instruction requires communication between microprocessor and memory or peripheral devices.
- The 8085 µP operations are synchronized by clock pulses and each instruction will require certain number of clock pulses for its execution.

- The knowledge about instruction cycle, machine cycle and T-state is essential to determine the time required for completing execution of an instruction or an operation in terms of clock pulses.

Instruction Cycle, Machine Cycle and T-State

- **Instruction Cycle** is defined as the time required to complete the execution of an 8085 instruction.
- An instruction in 8085 μP could be 1-byte, 2-byte or 3-byte in size.
- Further, each instruction cycle consists of **one to six operations** (parts) such as opcode fetch, read and write operations.

- **Machine Cycle** is defined as the time required to complete one operation of an instruction.

- All the instructions will have at least one operation. If an instruction has only one operation, then the machine cycle and instruction cycle are same.

Instruction Cycle, Machine Cycle and T-State

- **T- State** is defined as one sub-division of the operation performed in one clock period.
- Each sub-division or T-state of an operation is synchronized with the system clock and each T-state is precisely equal to one clock period.

- The terms T-state and clock period are often used synonymously.
- T-state or clock period is taken as the basic unit of operation time in 8085 µP.

- Execution of each operation or machine cycle will require specific number of T-states or clock period.

8085 µP Operations or Machine Cycles

- Execution of each 8085 µP instruction, requires performing various operations such Memory Read/Write, I/O Read/Write.
- The different **operations or machine cycles** present in 8085 µP are:

1. Opcode Fetch (OF)
2. Memory Read (MR)
3. Memory Write (MW)
4. I/O Read (IOR)
5. I/O Write (IOW)
6. Interrupt Acknowledge (IA)

- Each of the 74 instructions in 8085 µP Instruction set consists of one or more of these machine cycles.
- Each machine cycle is further divided into T-states

Examples of Instructions and Machine Cycles

Sl. No.	Instruction	No. of Machine Cycles	MC-1	MC-2	MC-3	MC-4
1	MOV A,B	1	OF	-	-	-
2	MVI A, 50H	2	OF	MR	-	-
3	LDA 5000H	4	OF	MR	MR	MR
4	STA 5000H	4	OF	MR	MR	MW
5	IN 80H	3	OF	MR	IOR	-
6	OUT 80H	3	OF	MR	IOW	-

- All instructions have at least one (Opcode Fetch) machine cycle

8085 µP Machine Cycle Timing Diagrams

- Each machine cycle requires specific T-states for its completion.
- The timing diagrams of each machine cycle represents the signals on various buses and pins of 8085 µP in relation to system clock pulse (T-state).

- The various signals represented in timing diagrams include
  - Clock Pulse or T-state (reference signal)
  - Address / Data Bus (parallel lines)
  - ALE (Address Latch Enable)
  - Status Signals (IO/<u>M</u>, S_1 and S_0)
  - Control Signals (<u>RD</u>, <u>WR</u> and <u>INTA</u>)

- A group of lines (Data / Address Bus) are represented by two parallel lines.

Opcode Fetch (OF) Machine Cycle

- The first operation in any instruction is Opcode Fetch. All the 8085 instructions will have one byte opcode and hence OF machine cycle is first operation of all the instructions.

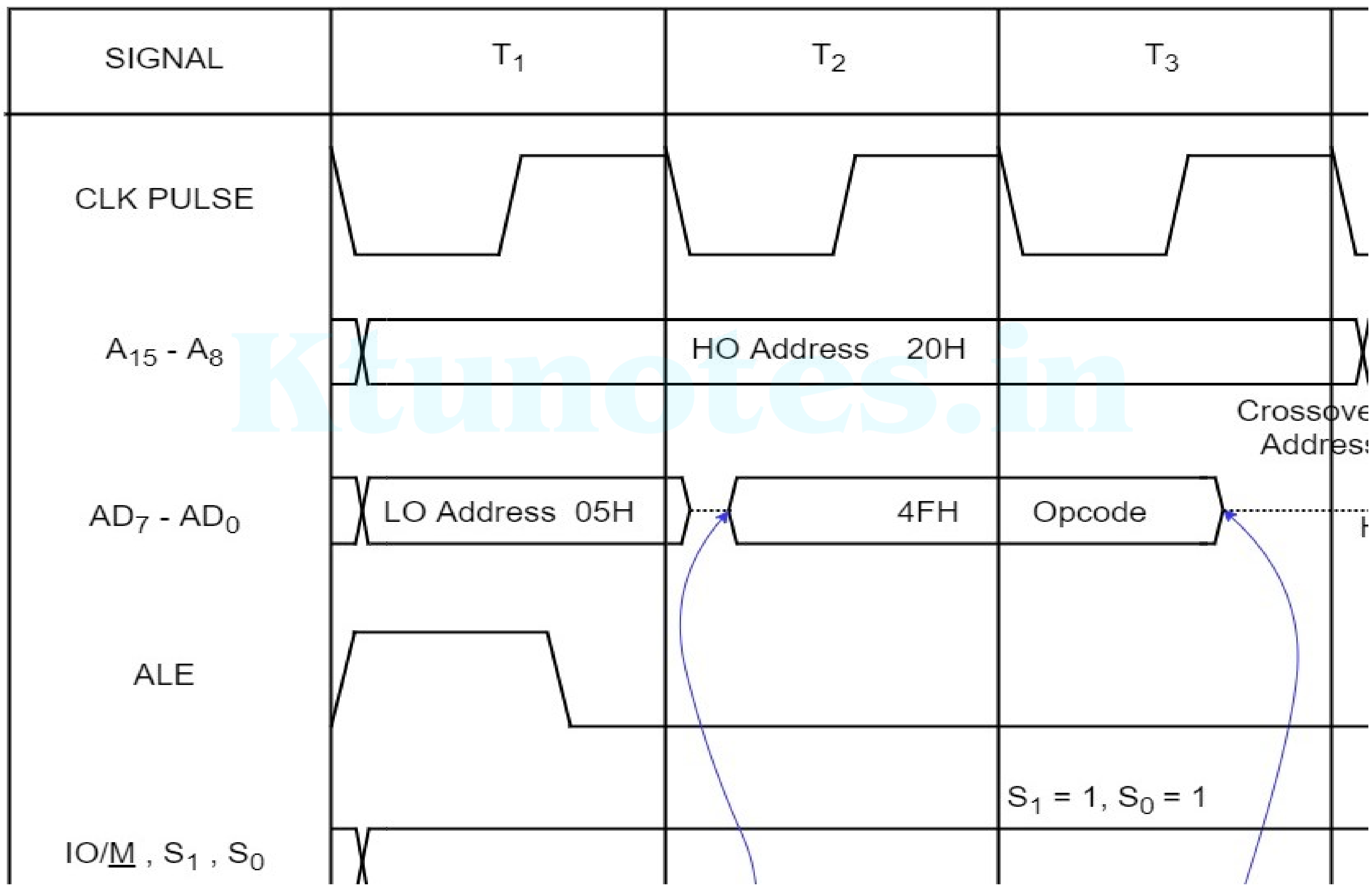
- Generally, the one byte instructions in 8085 Instruction set contains only opcode fetch machine cycle except some special cases.

- The OF cycle takes 4 T-states for completion. The first 3 T-states are used to fetch the opcode and the last T-state is used to decode and execute the opcode.

- Since the opcodes are stored in the R/W Memory, it is actually a Memory Read cycle.

- However, unlike OF cycle, the Memory Read cycle, requires only 3 T-states.

Timing Diagram of **Opcode Fetch** MC



Opcode Fetch (OF) Machine Cycle

- The address location of the opcode is assumed to be 2005_H and the opcode is assumed to be ($4F_H$ – MOV C,A).
- Different steps in OF machine cycle are given below

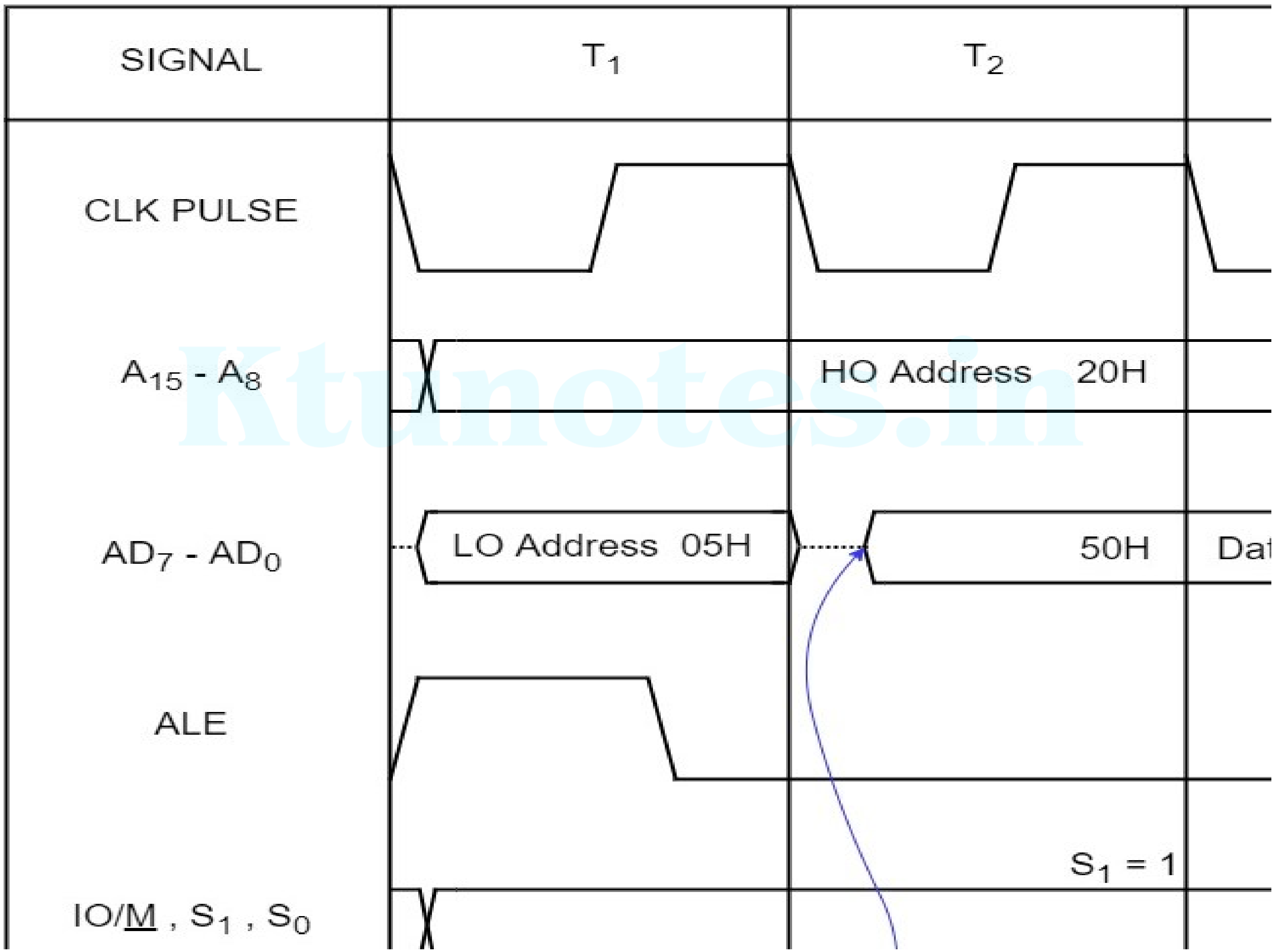
Sl. No.	T-State	Steps in OF Machine cycle
1	T_1	The PC places the HO 8-bits of the memory address on $A_{15} - A_8$ and LO 8-bits of the memory address on $AD_7 - AD_0$.
2		ALE signal is set HIGH and at the middle of T1 state, it goes LOW.
3		Status signals: $IO/\overline{M} = 0$, S1 =1 and S0 = 1. They remain unchanged.
4	T_2	The $\overline{RD}$ signal is set LOW to enable memory read and increment PC.
5		The opcode is placed on $AD_7 - AD_0$ from the memory location 2005_H.
6	T_3	The µP transfers the opcode on the A/D bus to Instruction Register (IR).
7		The µP makes the $\overline{RD}$ line HIGH to disable memory read.
8	T_4	The µP decodes the instruction and execute the commands.

Memory Read (MR) Machine Cycle

- A Memory Read machine cycle will be present if the 8085 µP instruction require reading data/operand from memory registers. The MR cycle will be present on all the 2-byte and 3-byte instructions.
- The MR cycle is similar to OF cycle as both reads hexadecimal numbers stored in memory registers.

- The MR cycle requires 3 T-sates for its completion.
- Example: MVI A, 50_H - Hex Codes in Memory : $3E_H$ 50_H
- In the above instruction, fetching first byte ($3E_H$) is OF cycle and fetching second byte (50_H) is a MR cycle.

Timing Diagram of **Memory Read** (MR) MC



Memory Read (MR) Machine Cycle

- The address location of the operand (50_H) is assumed to be 2005_H.
- Different steps in MR machine cycle are given below

Sl. No.	T-State	Steps in MR Machine cycle
1	T_1	The PC places the HO 8-bits of the memory address on $A_{15} - A_8$ and LO 8-bits of the memory address on $AD_7 - AD_0$.
2		ALE signal is set HIGH and at the middle of T1 state, it goes LOW.
3		Status signals: IO/$\underline{M}$ = 0, S1 =1 and S0 = 0. They remain unchanged.
4	T_2	The $\underline{RD}$ signal is set LOW to enable memory read and increment PC.
5		The operand / data is placed on $AD_7 - AD_0$ from the memory location 2005_H.
6	T_3	The µP stores the data on the A/D bus to Accumulator register according to the decoded instruction.
7		The µP makes the $\underline{RD}$ line HIGH to disable memory read.

Memory Write (MW) Machine Cycle

- A Memory Write machine cycle will be present if the 8085 µP instruction require storing data/operand in memory registers.

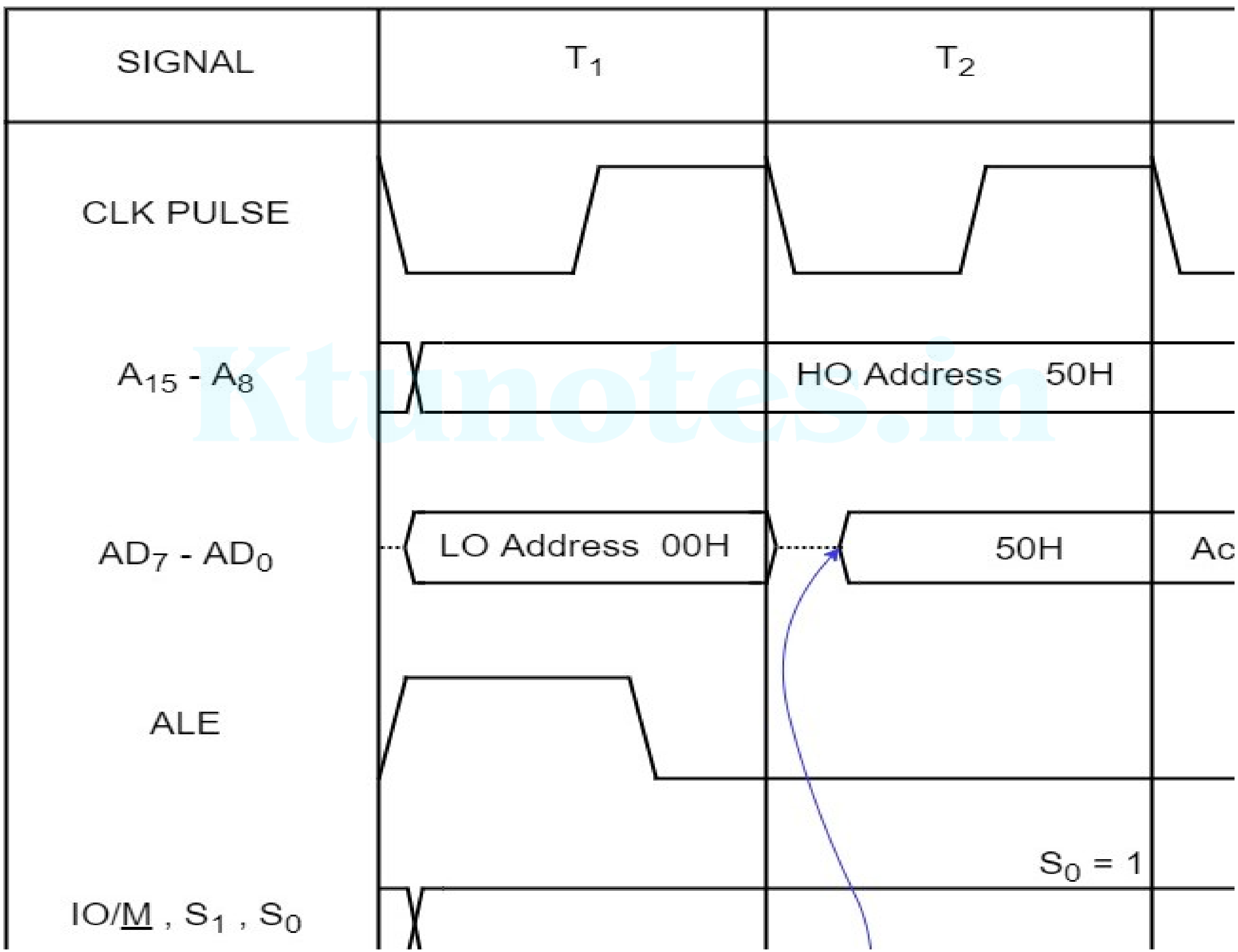
- The MW cycle differs from MR cycle only in the state of status signals and control signal ($\underline{WR}$ instead of $\underline{RD}$)

- The MW cycle requires 3 T-sates for its completion.

- Example: STA A, 5000_H - Hex Codes in Memory : 32_H 00_H 50_H

- In the above instruction, fetching first byte (33_H) is OF cycle. Fetching 2nd and 3rd bytes (00_H and 50_H) are MR cycles and storing the contents of Accumulator to memory register 5000_H is a MW cycle.

Timing Diagram of **Memory Write** (MW) MC



Memory Write (MW) Machine Cycle

- The address location of memory register is assumed to be 5000_H and the data in Accumulator to be stored is assumed to be 50_H.
- Different steps in MW machine cycle are given below.

Sl. No.	T-State	Steps in MR Machine cycle
1	T_1	The PC places the HO 8-bits of the memory address on $A_{15} - A_8$ and LO 8-bits of the memory address on $AD_7 - AD_0$.
2		ALE signal is set HIGH and at the middle of T1 state, it goes LOW.
3		Status signals: $IO/\underline{M}$ = 0, S1 =0 and S0 = 1. They remain unchanged.
4	T_2	The <u>WR</u> signal is set LOW to enable memory write and increment PC.
5		The operand / data is placed on $AD_7 - AD_0$ from the Accumulator register.
6	T_3	The µP store the data on the A/D bus to memory register 5000_H according to the decoded instruction.
7		The µP makes the <u>WR</u> line HIGH to disable memory write.

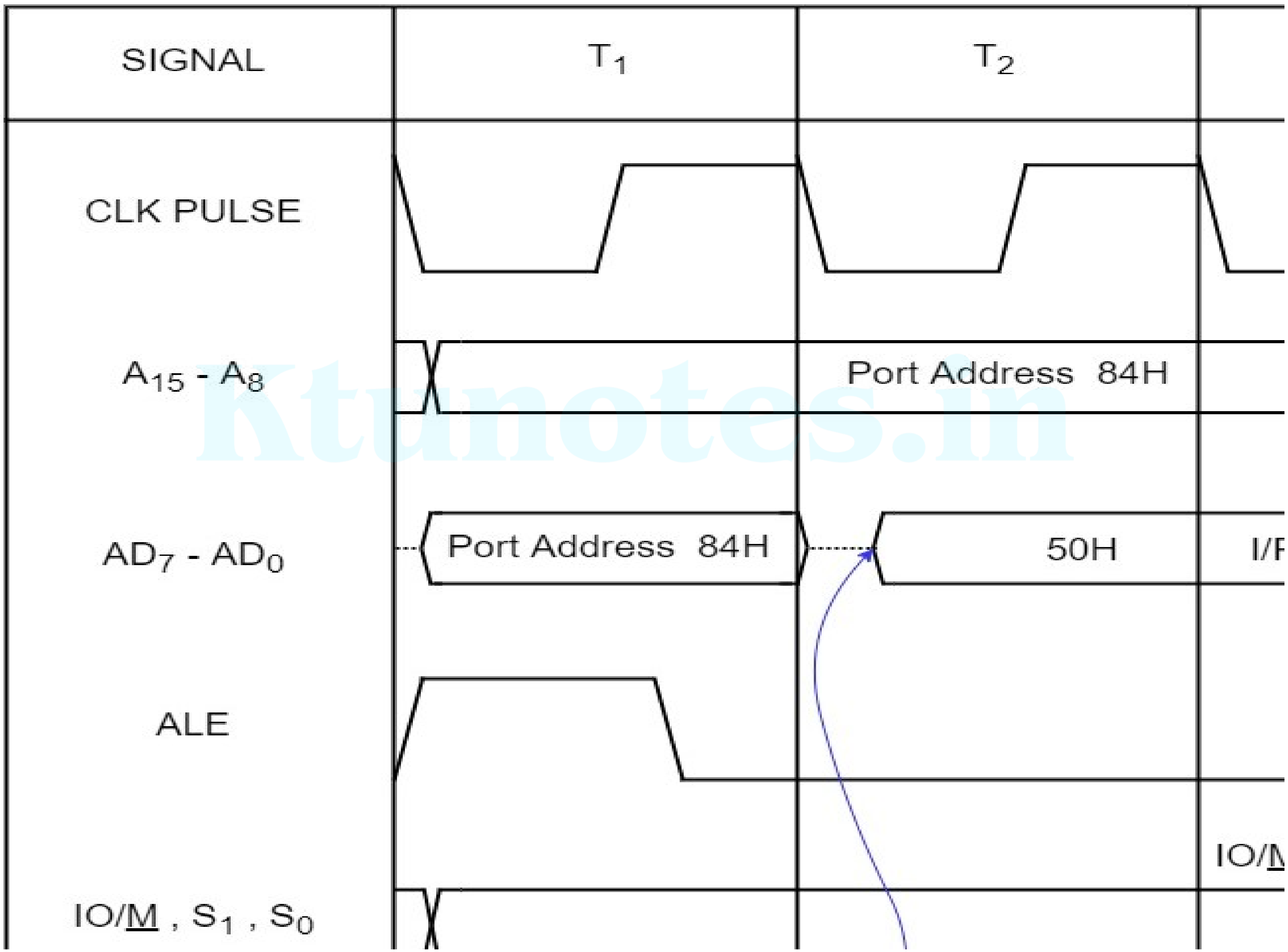
I/O Read (IOR) Machine Cycle

- I/O Read machine cycle will be present if the 8085 µP instruction require reading data/operand from input devices.
- The data transfer between 8085 µP and the peripheral devices is completed using the instructions IN/OUT.

- The IN/OUT instructions are used if the interfacing is done using peripheral mapped I/O (8-bit identification).
- The IN instruction consists of IOR cycle and the IOR cycle takes 3 T-states for its completion.

- Example : IN 84_H - Hex Codes in Memory : DB_H 84_H
- In the above instruction, fetching first byte (DB_H) is OF cycle. Fetching 2nd byte (84_H) is a MR cycle and storing data at the input port 84_H into the Accumulator is an IOR cycle.

Timing Diagram of **I/O Read** (IOR) MC



I/O Read (IOR) Machine Cycle

- The port address of input device is assumed to be 84_H and the data from the selected input device to be stored in Accumulator is assumed to be 50_H.
- Different steps in IOR machine cycle are given below.

Sl. No.	T-State	Steps in IOR Machine cycle
1		The PC places the port address of the I/O port specified in the instruction on both $A_{15} – A_8$ and $AD_7 – AD_0$.
2	T_1	ALE signal is set HIGH and at the middle of T1 state, it goes LOW.
3		Status signals: $IO/\overline{M}$ = 1, S1 =1 and S0 = 0. They remain unchanged.
4	T_2	The $\overline{RD}$ signal is set LOW to enable I/O read.
5		The input data from the connected device is placed on $AD_7 – AD_0$. (50_H)
6	T_3	The µP store the data on the A/D bus to Accumulator according to the decoded instruction.
7		The µP makes the $\overline{RD}$ line HIGH to disable I/O Read.

I/O Write (IOW) Machine Cycle

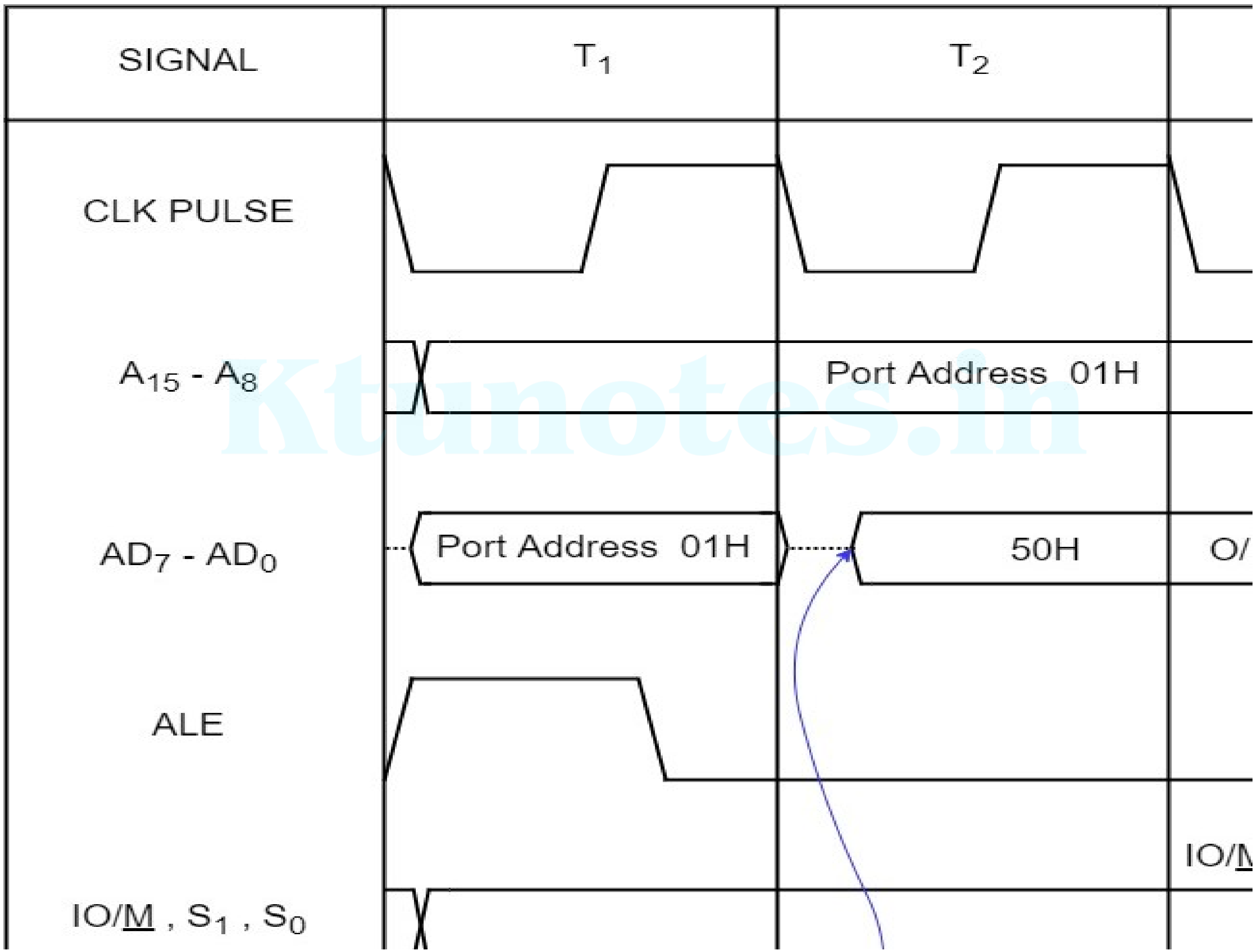
- I/O Write machine cycle will be present if the 8085 µP instruction require writing data/operand to output devices.

- The OUT instruction is used for sending data to interfaced output devices and it consists of IOW cycle
- The IOW cycle takes 3 T-states for its completion.

- Example : OUT 01_H - Hex Codes in Memory : $D3_H$ 01_H

- In the above instruction, fetching first byte ($D3_H$) is OF cycle. Fetching 2nd byte (01_H) is a MR cycle and sending data to the output port 01_H from Accumulator is an IOW cycle.

Timing Diagram of **I/O Write** (IOW) MC



I/O Write (IOW) Machine Cycle

- The port address of input device is assumed to be 01_H and the data send to the selected output device from Accumulator is assumed to be 50_H.
- Different steps in IOW machine cycle are given below.

Sl. No.	T-State	Steps in IOW Machine cycle
1	T_1	The PC places the port address of the I/O port specified in the instruction on both $A_{15} - A_8$ and $AD_7 - AD_0$.
2		ALE signal is set HIGH and at the middle of T1 state, it goes LOW.
3		Status signals: $IO/\overline{M} = 1$, S1 =0 and S0 = 1. They remain unchanged.
4	T_2	The $\overline{WR}$ signal is set LOW to enable I/O write.
5		The contents of Accumulator is placed on $AD_7 - AD_0$. (50_H)
6	T_3	The µP sends the data on the A/D bus to the output port specified in the decoded instruction.
7		The µP makes the $\overline{WR}$ line HIGH to disable I/O Write.

Machine Cycles of 8085 Instructions

- Generally, any 8085 µP instruction consists of one or more of the above discussed machine cycles.
- The opcode fetch cycle will be present in all the instructions and it will be the first machine cycle.

- Use the following steps to draw the timing diagram of a given instruction :
  1. Identify the opcode and operand of the given instruction
  2. Find the byte size of the instruction
  3. Identify the type of instruction and its function
  4. Based on the above steps identify the required machine cycles to complete the instruction execution.
  5. Calculate the no. of machine cycles and total no of T-states required for the timing diagram.
  6. Identify the control and status signals required for each machine cycle.
  7. Identify the memory/port addresses and data that need to be passed on to address bus and data bus respectively.

Machine Cycles of 8085 Instructions

- Let us take an example of OUT instruction
- Example: OUT 01H Hex-codes : D3 (2050H) 01 (2051H)

1. Opcode – D3 and Operand – 01
2. 2-byte instruction – 2 memory locations
3. Instruction Type – Data Transfer ; Function – Send data in accumulator to output
4. Required Machine Cycles : OF cycle, MR cycle and IOW cycle
5. No of MC – 3 ; Total No of T-States – 4+3+3 = 10
6. Control Signals : MEMR and IOW ;
7. Port Address – 01 ; Memory Address – 2050 and 2051 Data – Contents of Accumulator

Machine Cycles of 8085 Instructions